

Multi-Rail Isolated Power Supply Monitoring System

Capstone Design Documentation

Michael Capotosto

September 15, 2026

Rev. ET450-W1

Contents

1	Introduction	1
1.1	Project Overview	1
1.2	Project Motivation	2
1.3	System Architecture	2
1.4	Design Objectives	2
1.5	Project Scope	3
1.6	Report Organization	4
2	Preliminary Design Review (Week 1 Assignment)	5
2.1	Purpose and Scope	5
2.2	Major Functional Blocks	5
2.2.1	Functional Block Diagram	7
2.3	Preliminary Engineering Calculations	8
2.3.1	Current-Sense Shunt Resistor Sizing	8
2.3.2	PCB Trace and Copper-Pour Current Capacity	9
2.3.3	I ² C Pull-Up Resistor Sizing	10
2.3.4	LTC2945 Voltage Measurement Resolution	10
2.3.5	Negative-Rail Sign Convention	10
2.3.6	Alarm-Limit Scaling	10
2.3.7	Controller-Supply Power Budget	11
2.4	Preliminary Design Schematics	11
2.4.1	Sensor Board Schematic	11
2.4.2	Arduino Shield Board Schematic	19
2.5	Software Flowcharts	21
2.5.1	Arduino GIGA Firmware Flowchart	22

2.5.2	PSC and EPICS Communication Flow	24
2.5.3	Software Configuration Management	24
2.6	Preliminary Component List / Bill of Materials	24
2.6.1	Sensor Board Bill of Materials	25
2.6.2	Arduino Shield Board Bill of Materials	26
3	Installing EPICS Base and IOCs	27
3.1	Introduction	27
3.2	EPICS Installation Overview	27
3.3	Installing EPICS Base and Required Modules	27
3.4	Installing the PSU Monitor IOC	29
4	CS-Studio - Phoebus	31
4.1	Introduction	31
4.2	Installing Phoebus	31
5	Prototyping and Development	33
5.1	Introduction	33
5.2	Testing the LTC2945	33
5.2.1	Calculating Shunt Value for Test	34
5.2.2	Pictures of Evaluation Board, Display, and Phoebus Panels	35

Chapter 1

Introduction

1.1 Project Overview

The Multi-Rail Isolated Power Supply Monitoring System is a hardware and software system developed to provide centralized monitoring of multiple electrically isolated power-supply rails. The system measures both output voltage and load current, provides configurable alarm limits, displays operating status and measurement data via a local display or ethernet for integration with an EPICS-based control system.

The completed system monitors six independent power-supply rails:

- +5VA
- +5VB
- +5V
- -5V
- +15V
- -15V

Because the monitored supplies may be electrically isolated from one another, the monitoring system must preserve that isolation while still allowing all six channels to communicate with a common controller. Each supply rail therefore uses an independent voltage/current monitoring channel and an isolated digital communication path.

The primary controller is an Arduino GIGA R1 WiFi. The controller acquires measurement data from the individual monitor channels, converts the data to engineering units, applies the appropriate rail polarity, evaluates configured alarm limits, and presents the resulting information on a GIGA Display Shield. An Ethernet interface provides network communication between the embedded controller and an EPICS Input/Output Controller (IOC). The EPICS IOC then makes the measurements, alarms, and system status available to a CS-Studio Phoebus operator interface.

1.2 Project Motivation

Power-supply systems containing multiple isolated voltage rails present a monitoring challenge because each circuit associated with one supply can't share a direct electrical reference with another supply. Connecting conventional measurement or communication circuits across these references can break isolation between the rails.

A centralized monitor is desirable because it allows the user to observe all supply voltages and currents from a single interface, identify abnormal operating conditions, detect communication or sensor faults, and integrate the power-supply status into a larger control system.

The design developed for this project addresses these requirements by performing voltage and high-side current measurement locally on each monitored rail and transferring the resulting information across electrically isolated digital interfaces. The controller receives measurement data from all six rails without requiring their power references to be directly connected together.

1.3 System Architecture

The monitoring system is divided into three primary functional areas: the power-supply monitoring hardware, the microcontroller, and the supervisory control interface.

The power supply monitoring hardware contains the current-sense resistors, LTC2945 PMICs, and digital isolation circuitry. Each LTC2945 measures the supply voltage and the differential voltage developed across a high-side current-sense resistor. The resulting digital measurement information is sent over the I²C bus while maintaining the required electrical isolation.

The Arduino GIGA R1 WiFi provides the mcu processing functions. Its firmware initializes the monitoring channels, acquires voltage and current measurements, detects communication errors, converts raw measurements to engineering units, applies channel-specific polarity and scaling, evaluates alarm limits, and updates the local display. Configuration information, including network parameters and alarm limits, is retained in persistent storage so that required settings can survive a controller reset or power cycle.

Network communication is provided through an Ethernet interface using the PSC protocol. The EPICS IOC communicates with the embedded controller using PSCDriver and converts the transmitted values into EPICS process variables. CS-Studio Phoebus provides the remote operator interface for viewing measurements, alarm conditions, connection status, and system state.

1.4 Design Objectives

The major design objectives for the Multi-Rail Isolated Power Supply Monitoring System are to:

1. Measure the voltage of six independent power-supply rails.
2. Measure the load current of each rail using high-side current sensing.
3. Preserve electrical isolation between independently referenced power-supply rails and the common controller.
4. Convert acquired data into meaningful engineering units and correctly represent both positive and negative voltage rails.
5. Detect I²C communication failures and prevent invalid sensor data from being presented as valid measurements.
6. Compare voltage and current measurements against configurable upper and lower alarm limits.
7. Provide a local graphical display of voltage, current, alarm status, communication status, and network state.
8. Retain configuration and alarm settings through controller resets and power cycles.
9. Provide Ethernet communication for remote monitoring and configuration.
10. Integrate the monitoring system with an EPICS IOC and CS-Studio Phoebus operator interface.
11. Maintain the hardware, firmware, and control-system software under revision control so that changes made during design, implementation, and testing remain traceable.

1.5 Project Scope

The scope of the project includes the design and implementation of the sensor interface hardware, Arduino GIGA controller hardware, embedded firmware, Ethernet communication interface, EPICS IOC, and operator-interface software. The hardware portion includes the six voltage/current monitoring channels, current-sense resistors, digital isolation circuitry, controller interface, Ethernet circuitry, and supporting power-supply circuitry.

The software portion includes the embedded acquisition and alarm-processing firmware, persistent configuration storage, local display interface, PSC network communication, EPICS IOC configuration and database records, and the CS-Studio Phoebus operator interface.

Design files and software associated with the project are maintained in the project Git repository:

https://github.com/capotosto/PS_Monitor

The repository provides revision control for the hardware design, firmware, EPICS IOC files, documentation, and other project artifacts generated during development.

1.6 Report Organization

This report documents the development of the monitoring system from preliminary design through implementation and testing. The preliminary-design chapter defines the major functional blocks, presents the system architecture, documents supporting engineering calculations, includes the hardware schematics and bills of materials, and describes the primary firmware and control-system software flows.

Subsequent chapters document prototyping and development activities, the EPICS IOC implementation, and the CS-Studio Phoebus operator interface. Supporting project-management and design documents developed during the earlier phases of the capstone project are retained as part of the overall project documentation and provide traceability between the original requirements, design decisions, implementation, and final system.

Chapter 2

Preliminary Design Review (Week 1 Assignment)

2.1 Purpose and Scope

This chapter documents the preliminary hardware and software design of the Multi-Rail Isolated Power Supply Monitoring System. The design monitors six power-supply rails, measures rail voltage and load current, preserves electrical isolation between floating supplies and the controller, evaluates measurements against user-configurable alarm limits, presents local status on the controller display, and publishes measurements and status to an EPICS control-system interface.

This preliminary design is intended to establish the major analog and digital functional blocks, document supporting engineering calculations, define the hardware schematic base-line, and provide software flowcharts detailed enough to support implementation. Simulation and additional verification results can be added in the subsequent design phase.

2.2 Major Functional Blocks

The major system functions are summarized below.

1. **Power-Supply Interface:** Connect to the six monitored rails and route each rail through the associated voltage/current monitoring channel.
2. **Current Measurement:** Convert load current to a differential voltage across a precision shunt resistor and digitize it with the LTC2945.
3. **Voltage Measurement:** Measure the monitored rail voltage using the LTC2945 supply-voltage ADC channel.
4. **Electrical Isolation:** Maintain galvanic isolation between each floating monitored supply and the common controller-side I²C bus.

5. **Digital Acquisition:** Poll each monitor, identify communication faults, apply channel-specific sign/scaling information, and calculate engineering-unit values.
6. **Alarm Processing:** Compare voltage and current values against configured upper and lower limits and generate channel alarm states.
7. **Local User Interface:** Present all rail measurements, alarms, network state, and communication status on the Arduino GIGA Display.
8. **Network/EPICS Interface:** Publish measurements, alarm limits, alarm state, and connection status over Ethernet for the EPICS IOC and Phoebebus interface.

2.2.1 Functional Block Diagram

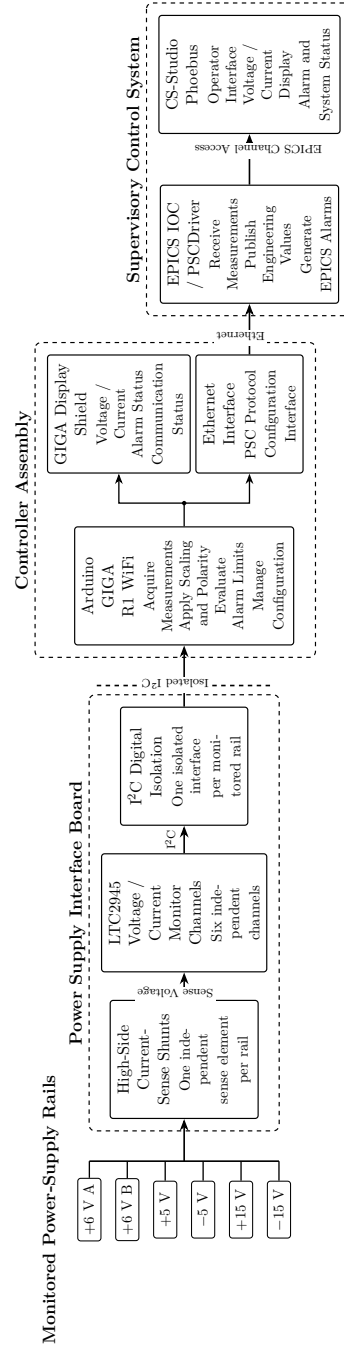


Figure 2.1: Preliminary functional block diagram of the multi-rail isolated power supply monitoring system. Six independent power-supply rails are measured using high-side current sensing and LTC2945 voltage/current monitoring channels. Digital isolation preserves electrical independence between the monitored supplies and the common controller. The Arduino GIGA processes the measurements, provides a local operator display, and transmits measurement and status information over Ethernet to the EPICS IOC and CS-Studio Phoebe operator interface.

2.3 Preliminary Engineering Calculations

2.3.1 Current-Sense Shunt Resistor Sizing

The LTC2945 measures the differential voltage between SENSE+ and SENSE– with a full-scale range of 102.4 mV. The ideal shunt resistance for a specified maximum measurable current is therefore

$$R_{SHUNT,ideal} = \frac{V_{SENSE,FS}}{I_{MAX}} = \frac{102.4 \text{ mV}}{I_{MAX}}. \quad (2.1)$$

To prevent saturation, a scaling factor, K_H , is applied to leave some headroom:

$$R_{SHUNT} = K_H \frac{102.4 \text{ mV}}{I_{MAX}}, \quad 0 < K_H < 1. \quad (2.2)$$

For example, $K_H = 0.80$ reserves 20% headroom for overload, tolerance, and transient conditions.

The shunt power at maximum continuous current is

$$P_{SHUNT} = I_{MAX}^2 R_{SHUNT}. \quad (2.3)$$

The installed resistor power rating should exceed this value.

2.3.1.1 Current Measurement Resolution

The LTC2945 current-sense ADC has a nominal sense-voltage resolution of 25 μV per LSB. The current represented by one ADC count is

$$I_{LSB} = \frac{25 \mu\text{V}}{R_{SHUNT}}. \quad (2.4)$$

This relationship provides an important design tradeoff: a smaller shunt lowers power dissipation but also increases the minimum current increment represented by one ADC count.

2.3.1.2 Production Shunt Design Table

Table 2.1: Production current-sense resistor calculations.

Rail	I_{MAX} (A)	R_{ideal} (Ω)	Selected R (Ω)	V_{SHUNT} (mV)	I_{LSB} (mA/LSB)	P_{MAX} (W)
+5VA	2.00	0.0512	0.039	78.0	0.641	0.156
+5VB	2.00	0.0512	0.039	78.0	0.641	0.156
+5V	1.00	0.1024	0.050	50.0	0.500	0.050
-5V	1.00	0.1024	0.050	50.0	0.500	0.050
+15V	1.00	0.1024	0.050	50.0	0.500	0.050
-15V	1.00	0.1024	0.050	50.0	0.500	0.050

2.3.1.3 Evaluation-Board Shunt Calculation

The prototype LTC2945 evaluation board was tested from a 5 V source with a load range of approximately 1 k Ω to 10 k Ω . This is a low-current bench-test condition and is separate from the production shunt design.

At the minimum load resistance, the shunt required to place the current-sense input at full scale is

$$R_{S,FS} = \frac{V_{SHUNT}R_L}{V_S - V_{SHUNT}} = \frac{(0.1024 \text{ V})(1 \text{ k}\Omega)}{5 \text{ V} - 0.1024 \text{ V}} = 20.91 \Omega. \quad (2.5)$$

Applying approximately 15% headroom gives

$$R_S = (0.85)(20.91 \Omega) \approx 17.8 \Omega. \quad (2.6)$$

At a 10 k Ω load,

$$I_{MIN} = \frac{5 \text{ V}}{10 \text{ k}\Omega + 17.8 \Omega} \approx 499.1 \mu\text{A}, \quad (2.7)$$

and therefore

$$V_{SHUNT,MIN} = I_{MIN}R_S \approx (499.1 \mu\text{A})(17.8 \Omega) \approx 8.88 \text{ mV}. \quad (2.8)$$

The prototype sense voltage therefore remains inside the LTC2945 102.4 mV full-scale range over the intended test loads.

2.3.2 PCB Trace and Copper-Pour Current Capacity

Power-path copper must carry the maximum rail current without excessive temperature rise or voltage drop. KiCad's track-width calculator uses a closed-form fit to IPC-2152 data. For an external trace it can be expressed as

$$\Delta T = KI^a W^b T_h^c, \quad (2.9)$$

where I is current in amperes, W is trace width in mils, T_h is copper thickness in mils, and for external tracks the KiCad coefficients are

$$K = 215.3, \quad a = 2, \quad b = -1.15, \quad c = -1.0. \quad (2.10)$$

Solving for minimum width,

$$W_{MIN} = \left(\frac{\Delta T}{KI^a T_h^c} \right)^{1/b}. \quad (2.11)$$

For 1 oz copper ($T_h \approx 1.378$ mil) and a preliminary 10 °C allowed rise, the calculated external-trace widths are approximately 3.27 mil at 0.5 A, 10.9 mil at 1 A, and 36.4 mil at 2 A. These are thermal minimums only; the actual layout should use wider traces or copper pours where space permits.

2.3.3 I²C Pull-Up Resistor Sizing

The I2C bus buffer and I2C line driver both have internal active current sources that eliminate the need for external pullups. Footprints were placed for 4.7k pullups but are set as Do Not Populate.

2.3.4 LTC2945 Voltage Measurement Resolution

The LTC2945 rail-voltage channel has a nominal full-scale range of 102.4 V and a nominal resolution of 25 mV per LSB. The quantization-only percentage of a measured rail voltage is therefore

$$\epsilon_Q = \frac{25 \text{ mV}}{V_{RAIL}} \times 100\%. \quad (2.12)$$

For example, one count corresponds to approximately 0.42% of a 6 V rail, 0.50% of a 5 V rail, and 0.17% of a 15 V rail. This is only ADC resolution and not total measurement accuracy.

2.3.5 Negative-Rail Sign Convention

The -5 V and -15 V supplies are electrically isolated from the controller. The monitor can therefore be placed in the high side of each isolated rail and read a positive local differential quantity even when the system-level rail is labeled negative. Firmware applies the configured sign after acquisition:

$$V_{DISPLAY} = S_V V_{MEASURED}, \quad S_V \in \{+1, -1\}. \quad (2.13)$$

The same channel metadata is used by the local display and network interface so that a negative rail is represented consistently throughout the system.

2.3.6 Alarm-Limit Scaling

User alarm thresholds are stored and compared in engineering units. For a lower voltage limit V_L and upper voltage limit V_H , the normal voltage state is

$$V_L \leq V_{MEAS} \leq V_H. \quad (2.14)$$

Similarly, for lower and upper current limits,

$$I_L \leq I_{MEAS} \leq I_H. \quad (2.15)$$

If raw ADC thresholds are ever written directly to the LTC2945, the corresponding register value must be calculated from the ADC LSB size. For the current-sense channel,

$$N_I = \frac{I_{LIMIT} R_{SHUNT}}{25 \mu\text{V}/\text{LSB}}, \quad (2.16)$$

and for the rail-voltage channel,

$$N_V = \frac{V_{LIMIT}}{25 \text{ mV}/\text{LSB}}. \quad (2.17)$$

2.3.7 Controller-Supply Power Budget

A preliminary controller power budget was developed using datasheet maximum values where available and conservative design-current allowances for assemblies where a complete board-level maximum current is not specified. A 25% design margin is applied to the calculated load.

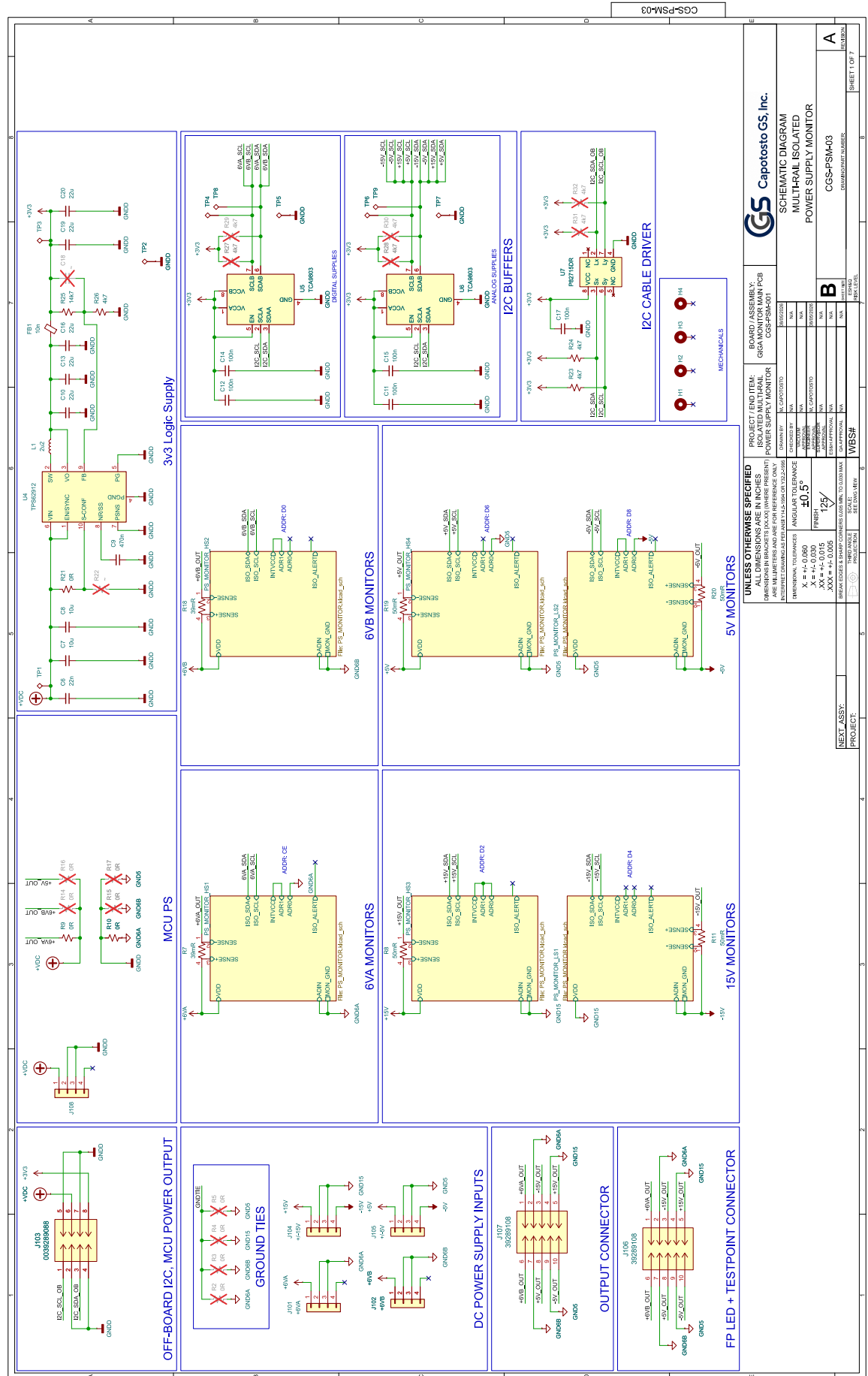
Table 2.2: Preliminary controller-side power budget.

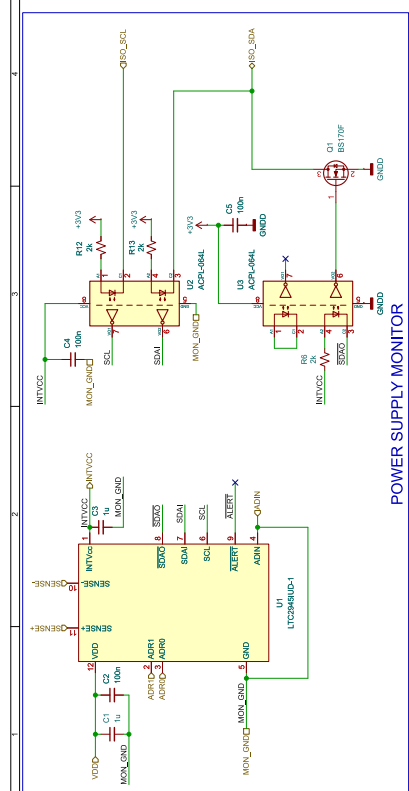
Load	Supply (V)	Design Current (A)	Power (W)
Arduino GIGA R1 WiFi	5.0	0.500	2.500
GIGA Display Shield	3.3	0.250	0.825
W6100 Ethernet interface	3.3	0.265	0.875
I ² C isolation/controller circuitry	3.3	0.065	0.215
Subtotal	–	–	4.415
25% design margin	–	–	1.104
Total	–	–	5.519

2.4 Preliminary Design Schematics

The preliminary hardware schematics are maintained in KiCad and are included below directly from the plotted PDF files in the project hardware directory.

2.4.1 Sensor Board Schematic



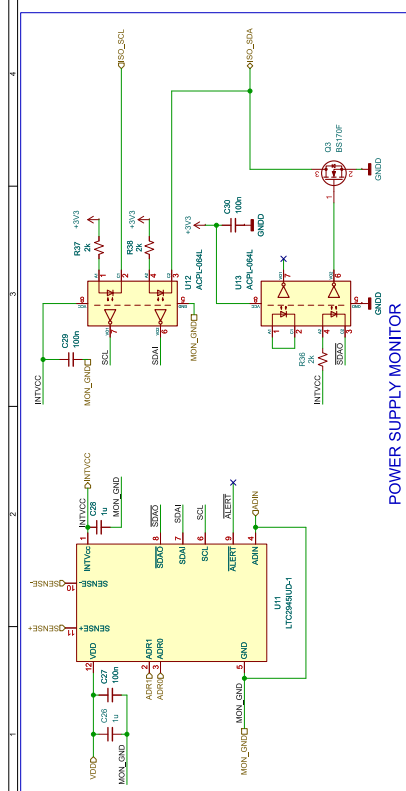


CGS-PSM-03

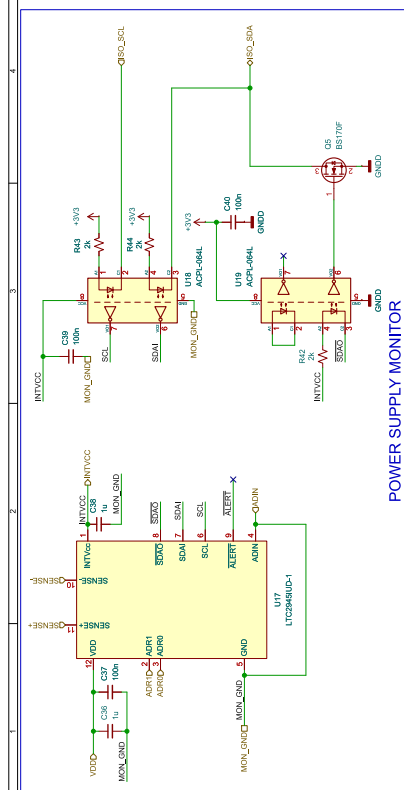
B CGS-PSM-03
DOCUMENT NUMBER
REV. 1.0
DATE: 10/15/2024
DESIGNED BY: J. J. J.

A SHEET 1 OF 7
REVISIONS

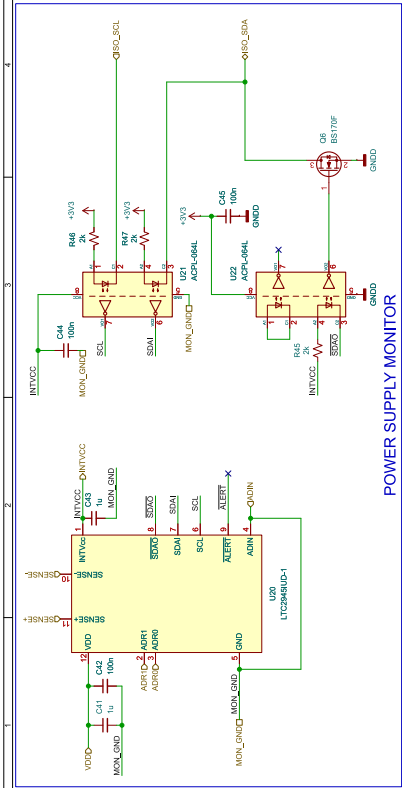








POWER SUPPLY MONITOR



POWER SUPPLY MONITOR

CGS-PSM-03

B	CGS-PSM-03 DOCUMENT NUMBER	A	SHEET 6 OF 7

2.4.2 Arduino Shield Board Schematic

2.5 Software Flowcharts

The software for the Multi-Rail Isolated Power Supply Monitoring System is maintained with the hardware design files in the project Git repository:

https://github.com/capotosto/PS_Monitor

The repository provides version control for the Arduino GIGA firmware, hardware-design files, and EPICS IOC software used by the monitoring system. The embedded firmware is maintained under the **firmware** portion of the repository, while the EPICS IOC and associated database/configuration files are maintained under **epics-ioc**. The flowcharts in this section describe the major software operations implemented by these portions of the project.

The Arduino GIGA firmware performs system initialization, acquires voltage and current measurements from the six LTC2945 monitor channels, evaluates alarm conditions, updates the local display, and publishes data through the Ethernet interface. The primary firmware execution sequence is:

1. Initialize the GIGA Display, persistent settings, Ethernet interface, I²C interface, and channel configuration.
2. Load stored network parameters and voltage/current alarm limits from persistent storage.
3. Initialize communication with each of the six LTC2945 monitoring channels.
4. Enter the continuous acquisition and communication loop.
5. Poll each LTC2945 channel for voltage and current measurements.
6. Verify I²C communication status for each channel.
7. If communication fails, mark the associated channel with an I²C error and prevent invalid measurement data from being used.
8. Convert valid raw measurement data to engineering units and apply the appropriate channel polarity and calibration/scaling information.
9. Compare each voltage and current measurement with its configured upper and lower alarm limits.
10. Update the GIGA Display when measurement values, alarm states, or communication states change.
11. Publish the current measurements and status information through the Ethernet PSC protocol interface.
12. Service incoming configuration requests and store modified settings when required.
13. Repeat the acquisition and communication sequence continuously.

2.5.1 Arduino GIGA Firmware Flowchart

Figure 2.2 shows the top-level execution flow of the Arduino GIGA firmware. Following initialization, the controller continuously acquires data from each monitoring channel, verifies communication, converts valid measurements to engineering units, evaluates alarm limits, updates the local display, and services the Ethernet/PSC interface.

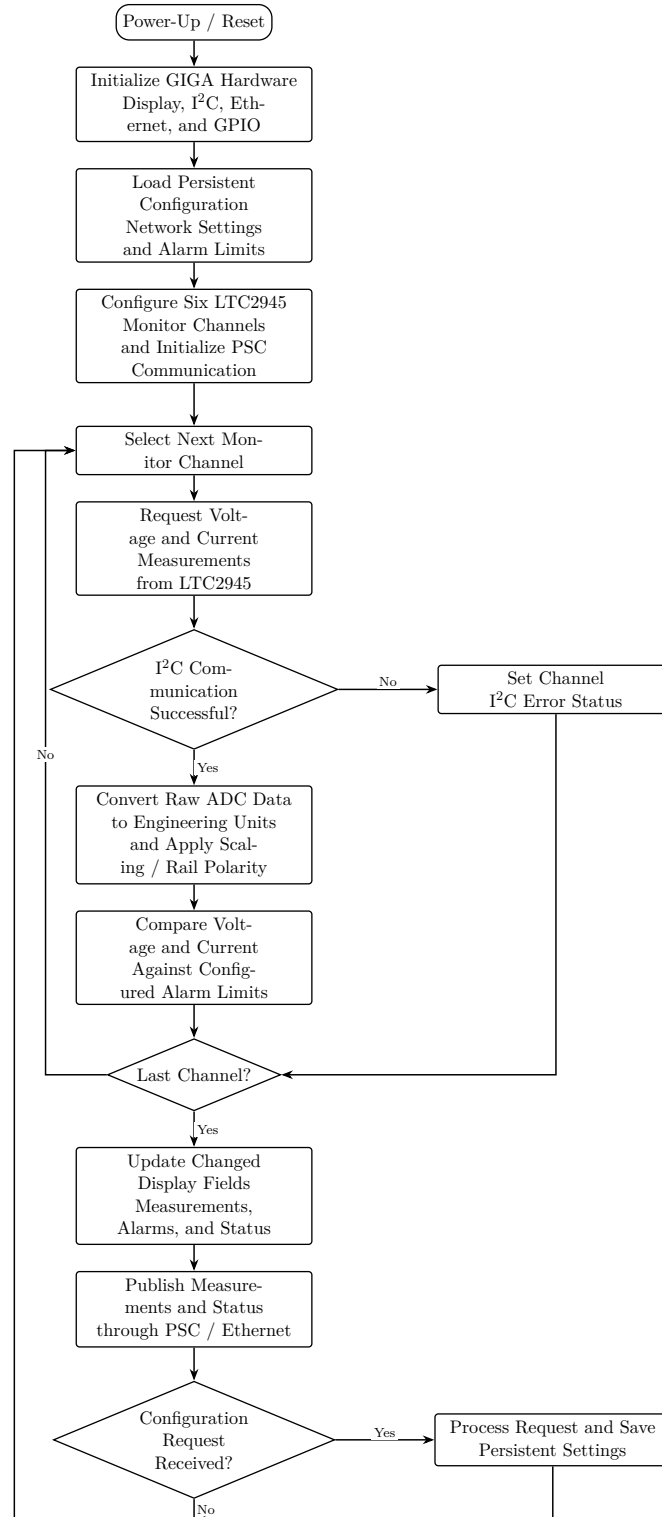


Figure 2.2: Top-level Arduino GIGA firmware flowchart. Following initialization, the controller continuously acquires measurements from the six LTC2945 monitoring channels, validates I²C communication, converts valid data to engineering units, evaluates configured alarm limits, updates the local display, publishes measurements through the PSC/Ethernet interface, and services configuration requests.

2.5.2 PSC and EPICS Communication Flow

The Arduino GIGA does not directly implement EPICS Channel Access. Instead, measurement and status information is transmitted through the Ethernet interface using the PSC protocol. The EPICS IOC uses PSCDriver to communicate with the GIGA, maps the received values into EPICS process variables, and makes those process variables available to the CS-Studio Phoebus operator interface.

This separation allows the embedded controller to remain responsible for hardware acquisition and local alarm evaluation while the IOC provides the interface between the embedded hardware and the facility control system. Figure 2.3 summarizes this software data path.

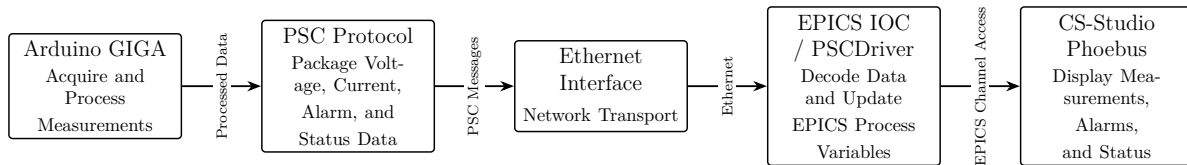


Figure 2.3: Software communication path from the Arduino GIGA firmware through PSC-Driver and the EPICS IOC to the CS-Studio Phoebus operator interface.

2.5.3 Software Configuration Management

Firmware, IOC configuration files, database definitions, and operator-interface files are maintained under Git version control so that hardware and software revisions can be associated with a defined project baseline. The current software implementation and revision history are available in the `PS_Monitor` repository at:

https://github.com/capotosto/PS_Monitor

Changes made during implementation and testing are committed to the repository, providing traceability between the preliminary design presented in this report and the final implemented system. The repository therefore serves as the controlled source for the firmware and control-system software associated with the project.

2.6 Preliminary Component List / Bill of Materials

The current bills of materials for the Sensor Board and Arduino Shield Board are maintained with their respective KiCad hardware-design files in the project `hardware` directory. They are included below for reference.

2.6.1 Sensor Board Bill of Materials

BOM: Power Sensor Board				Production		1 Tape and Reel/Reeling Fees All: No		Add 10% spares for components excluding modules			
Reference	Value	DNP	MPN	Manufacturer	DPN	Distributor	Qty	Qty+10%	Order Qty	Extended Price	
C1,C3,C21,C23,C26,C28,C31,U			CL10A105KB8NNNC	Samsung Electro-Mechanics	1276-1860-6-ND	Digkey	12	14	100	\$3.77	
C2,C4,C5,C11,C12,C14,C15,(100n			GRM155R61H104KE14D	Murata Electronics	490-10697-6-ND	Digkey	23	26	100	\$1.41	
C6	22n		GRM155R71H222KAQ01D	Murata Electronics	490-1305-6-ND	Digkey	1	2	100	\$1.01	
C7,C8	10u		C2012X751E106K125AC	TDK	445-181600-6-ND	Digkey	2	3	10	\$3.58	
C9	470n		CL1005XGR1E474K050BB	TDK	445-8020-6-ND	Digkey	1	2	100	\$3.17	
C10,C13,C16,C19,C20	22u		GMC21X7S226M10NT	Cal-Chip Electronics, Inc.	4713-GMC21X7S226M10NTDKR-ND	Digkey	5	6	15	\$11.30	
C18	-	DNP				Digkey	1	2			
F81	10n		BLE18PS080SN1D	Murata Electronics	490-BLE18PS080SN1DDKR-ND	Digkey	1	2	10	\$2.30	
H1,H2,H3,H4	MountingHole_Pad						4	5			
J101	+6VA		0039299049	Molex	WM14518-ND	Digkey	1	2	1	\$2.70	
J102	+6VB		0039299049	Molex	WM14518-ND	Digkey	1	2	1	\$2.70	
J103	0039289088		0039289088	Molex	WM3835-ND	Digkey	1	2	1	\$4.58	
J104	+/-15V		0039299049	Molex	WM14518-ND	Digkey	1	2	1	\$2.70	
J105	+/-5V		0039299049	Molex	WM14518-ND	Digkey	1	2	1	\$2.70	
J106,J107	39289108		0039289108	Molex	23-0039289108-ND	Digkey	2	3	1	\$5.02	
J108	Conn_01x04		0039299049	Molex	WM14518-ND	Digkey	1	2	1	\$2.70	
L1	2u2		74438356022	Wurth Elektronik	732-8232-6-ND	Digkey	1	2	3	\$5.82	
Q1,Q2,Q3,Q4,Q5,Q6	BS170F		N7A7002NT1G	onsemi	N7A7002NT1GOSDKR-ND	Digkey	6	7	25	\$4.43	
R2,R3,R4,R5,R14,R15,R16,R17,OR		DNP		5108 Keystone Electronics	36-5108DKR-ND	Digkey	8	9			
R6,R12,R13,R33,R34,R35,R36,2k			ERL-2RKF2001X	Panasonic Industry	P2.00KLDKR-ND	Digkey	18	20	100	\$1.64	
R7,R18	39mR		LVK20R039FER	Ohmite	273-LVK20R039FERDKR-ND	Ohmite	2	3	5	\$9.45	
R8,R11,R19,R20	50mR		LVK20R050FER	Ohmite	273-LVK20R050FERDKR-ND	Ohmite	4	5	5	\$9.45	
R9,R10	0R			5108 Keystone Electronics	36-5108DKR-ND	Digkey	2	3	10	\$2.01	
R21	0R		CRCW0402000Z0ED	Vishay Dale	541-0.0IDKR-ND	Digkey	1	2	100	\$0.99	
R22	-	DNP				Digkey	1	2			
R23,R24	4k7		CRCW04024K70FKED	Panasonic Industry	541-4.70KLCI-ND	Digkey	2	3	100	\$1.57	
R25	14k7		ERA-2APB1472X	Panasonic Industry	P17731CI-ND	Digkey	1	2	10	\$4.52	
R26	4k7		ERA-2ARB472X	Panasonic Industry	P4.7KPCPI-ND	Digkey	1	2	10	\$4.99	
R27,R28,R29,R30,R31,R32	4k7	DNP	CRCW04024K70FKED	Vishay Dale	541-4.70KLCI-ND	Digkey	6	7			
TP1,TP3	5000		5000 Keystone		36-5000-ND	Digkey	2	3	10	\$2.04	
TP2	5001		5001 Keystone		36-5001-ND	Digkey	1	2	10	\$2.03	
TP4,TP5,TP6,TP7,TP8,TP9	5015		5015 Keystone		36-5015CI-ND	Digkey	6	7	25	\$8.29	
U1,U8,U11,U14,U17,U20	LTC2945IUD-1		LTC2945IUD#TRPBF	Analog Devices Inc.	LTC2945IUD#TRPBFDKR-ND	Digkey	6	7	10	\$110.97	
U2,U3,U9,U10,U12,U13,U15,LACPL-064L			ACPL-064L-500E	Broadcom Limited	516-2614-6-ND	Digkey	12	14	15	\$56.27	
U4	TPS62912		TPS62912RPUR	Texas Instruments	595-TPS62912RPUR	Mouser	1	2	3	\$7.77	
U5,U6	TCA9803		TCA9803DQKR	Texas Instruments	296-47829-6-ND	Digkey	2	3	10	\$6.84	
U7	P82B715DR		P82B715DR	Texas Instruments	296-24729-6-ND	Digkey	1	2	5	\$13.55	
TOTAL COST:										\$302.26	

2.6.2 Arduino Shield Board Bill of Materials

BOM: Arduino Shield Board				Production		1 Tape and Reel/Reeling Fees All: No		Add 10% spares for components excluding modules			
Reference	Value	DNP	MPN	Manufacturer	DPN	Distributor	Qty	Qty+10%	Order Qty	Extended Price	
A201	Arduino_Giga_R1_WiFi		ABX00063	Arduino	1050-ABX00063-ND	Digkey	1	2	1	\$69.60	
C6	22n	DNP	GRM155R71H22XA01D	Murata Electronics	490-1305-6-ND	Digkey	1	2	0		
C7,C8	10u	DNP	C2012X7S1E106K125AC	TDK	445-181600-6-ND	Digkey	2	3	0		
C9	470n	DNP	C1005X6R1E474K050BB		445-8020-6-ND	Digkey	1	2	0		
C10,C13,C16,C19,C20	22u	DNP	GMC21X7S226M10NT	Cal-Chip Electronics, Inc.	4713-QMC21X7S226M10NTDKR-ND	Digkey	5	6	0		
C17,C201,C202,C203,C211,C215,C216,C217,C220,C221,C223,C224,C225,C227	100n		GRM155R61H104KE14D	Murata Electronics	490-10697-6-ND	Digkey	14	16	100	\$1.41	
C218	3u3					Digkey	1	2			
C219	1n		1206GC102KAT1A	KYOCERA AVX	478-2956-6-ND	Digkey	1	2	10	\$1.43	
C222,C226	18p		GRM1555C1H180JA01D	Murata Electronics	490-5858-6-ND	Digkey	2	3	100	\$1.43	
F81	10n	DNP	BLE18PS080SN1D	Murata Electronics	490-BLE18PS080SN1DDKR-ND	Digkey	1	2			
FB201,FB202,FB203	120R		BBUP00160805121Y00	Pulse Electronics	553-BBUP00160805121Y00CT-NND	Digkey	3	4	25	\$1.97	
J201	SI-46014-F		SI-46014-F	Bel Fuse Inc.	5923-SI-46014-F-ND	Digkey	1	2	1	\$5.81	
J309	0039289088		0039289088	Molex	WM3835-ND	Digkey	1	2	1	\$4.58	
L1	2u2	DNP	74438356022	Wurth Elektronik	732-8232-6-ND	Digkey	1	2	3	\$5.82	
N101	Giga_Display		ASX00039	Arduino	1050-ASX00039-ND	Digkey	1	2	1	\$63.60	
N201	Connector_Set		85	Adafruit Industries	1528-1074-ND	Digkey	1	2	2	\$3.90	
R21	0R	DNP	ERJ-2RKF4701X	Panasonic Industry	P4.70KLDKR-ND	Digkey	1	2			
R23,R24,R205,R206,R207	4K7		CRCW04024K70FKED	Vishay Dale	541-4.70KLCF-ND	Digkey	5	6	100	\$1.57	
R25	15k2	DNP	ERJ-2RKF4701X	Panasonic Industry	P4.70KLDKR-ND	Digkey	1	2			
R26	4k87	DNP	ERJ-2RKF4701X	Panasonic Industry	P4.70KLDKR-ND	Digkey	1	2			
R31,R32	4k7	DNP	ERJ-2RKF4701X	Panasonic Industry	P4.70KLDKR-ND	Digkey	2	3			
R101,R102,R103,R104,R105	10k					Digkey	5	6			
R208	12k3		RN73R1ETTP1232B25	KOA Speer Electronics, Inc.	2019-RN73R1ETTP1232B25DKR-ND	Digkey	1	2	50	\$3.02	
R209,R210,R215,R216	49R9		CRCW040249R9FKED	Vishay Dale	541-49.9LDKR-ND	Digkey	4	5	100	\$1.57	
R211,R212,R213,R214	3R3		CRCW04023R30FKED	Vishay Dale	541-3.30LLDKR-ND	Digkey	4	5	100	\$2.03	
R217	1M		CRCW04021M00FKED	Vishay Dale	541-1.00MLDKR-ND	Digkey	1	2	100	\$1.57	
R218,R219	470R		ERJ-2RKF4700X	Panasonic Industry	P470LDKR-ND	Digkey	2	3	100	\$1.64	
R220	0R		CRCW04020000Z0ED	Vishay Dale	541-0.0DKR-ND	Digkey	1	2	100	\$0.99	
TP1,TP3		2 DNP		5000 Keystone	36-5000-ND	Digkey	2	3			
TP2		1 DNP		5001 Keystone	36-5001-ND	Digkey	1	2			
TP101		1		5000 Keystone	36-5000-ND	Digkey	1	2	10	\$2.04	
TP102		1		5001 Keystone	36-5001-ND	Digkey	1	2	10	\$2.03	
TP201,TP202,TP203		3		5015 Keystone	36-5015CF-ND	Digkey	3	4	10	\$3.70	
U4	TPSG2912	DNP					1	2			
U7	P82715DR		P82B715DR	Wiznet	296-24729-6-ND		1	2	2	\$5.42	
U202	W6100-Q		W6100Q	Wiznet	950-W6100Q	Wiznet	1	2	2	\$5.04	
Y201	25MHz		FL2500169	Diodes Incorporated	31-FL2500169DKR-ND	Digkey	1	2	2	\$1.38	
							TOTAL COST:			\$191.55	

Chapter 3

Installing EPICS Base and IOCs

3.1 Introduction

A simple IOC was written to interface the Arduino Giga which monitors the six supplies. The following section details how to install EPICS Base and required modules, and compile the IOC. It is written assuming a Linux environment; this particular build was done on Ubuntu 22.04LTS.

3.2 EPICS Installation Overview

EPICS installations performed by M. Capotosto loosely follow the below directory structure:

- `EPICS_BASE=/epics/base` or `/epics/base_[version]` or similar
- `SUPPORT=/epics/support` – or sometimes `MODULES` instead of `SUPPORT` is used.

Modules required for the PSU monitor are:

- `pscdrv`: Portable Streaming Controller Driver
- `epics/iocs` - each IOC is cloned into this repository
- `epics/systemd-softioc` - This is a utility for managing IOCs as a system service

3.3 Installing EPICS Base and Required Modules

1. Install Dependencies for Build Install git, build-essential, and perl:

```
sudo apt -y install git build-essential perl libtirpc-dev re2c libevent-dev
autoconf automake libtool pkg-config
```

2. Create the EPICS user group, and the empty EPICS directory:

```
sudo groupadd epics
sudo usermod -aG epics $USER
newgrp epics
sudo mkdir -p /epics
sudo chown $USER:epics /epics
sudo chmod 2775 /epics
```

3. EPICS Base is cloned into /epics/base via:

```
git clone -recursive -b 7.0
https://github.com/epics-base/epics-base.git /epics/base
cd /epics/base
```

4. Build EPICS:

```
make clean
make
```

5. Create the remaining directory structure:

```
mkdir -p /epics/iocs /epics/modules
```

6. Clone PSC Driver:

- (a) `cd /epics/modules`
- (b) `git clone https://github.com/mdavidsaver/pscdrv.git pscdrv`
- (c) `cd /epics/modules/pscdrv`
- (d) Set up required files in `configure/RELEASE`, `configure/RELEASE.LOCAL`
- (e) Build with `make clean`, and `make`

7. Clone Manage IOCs Utility:

- (a) `cd /epics`
- (b) `git clone https://github.com/NSLS2/systemd-softioc.git`
- (c) Navigate to the repo directory `cd systemd-softioc`
- (d) Make the install script executable and run:


```
sudo chmod +x install.sh
sudo ./install.sh
sudo usermod -aG epics softioc
newgrp
sudo chown softioc:epics /epics/iocs
sudo chmod 2775 /epics/iocs
```

3.4 Installing the PSU Monitor IOC

1. Change to IOC directory `cd /epics/iocs`
2. Clone the repo:
`git clone https://github.com/capotosto/PS_Monitor psuMonitorIOC`
3. `cd psuMonitorIOC`
4. Create the `RELEASE.local` file, `touch configure/RELEASE.local`
5. `echo -e "EPICS_BASE = /epics/base\nPSCDRV = /epics/modules/pscdrv\n"`

6. Build the IOC: `make clean, make`
7. Update the `st.cmd` file as needed: `cd iocBoot/iocpsuMonitor`
8. Create the `st.cmd` file used by `systemd-softioc`:

```
echo -e '#!/bin/bash\n\n
cd "/epics/iocs/psuMonitorIOC/iocBoot/iocpsuMonitor"\n./st.cmd' > st.cmd
```

9. Make the new `st.cmd` executable: `chmod +x st.cmd`
10. Find the next available port: `manage-iocs nextport`
11. Configure this in the `config` file: `nano config`
12. Set the `NAME=`, `PORT=` your next port, `USER=softioc`, and `HOST=$HOSTNAME`
13. Update the directory path in the root-level `st.cmd` to the `iocBoot/iocpsuMonitor/st.cmd`
14. Enroll in `manage-iocs` as a service: `sudo manage-iocs install psuMonitorIOC`
15. Enable auto-start on boot: `sudo manage-iocs enable psuMonitorIOC`
16. Check the status, it should be running! `manage-iocs status`
17. And if you want other information on the ioc: `manage-iocs report`

Chapter 4

CS-Studio - Phoebus

4.1 Introduction

CS-Studio Phoebus is installed using precompiled binaries. This application requires a Java runtime environment be installed.

The 'BOB' files - the user interface panels, are located in the IOC's directories, `/epics/iocs/PSCtest/gui` and `/epics/iocs/zpsc_ioc/gui`

Behavior of some features of Phoebus must be modified by the shell script used to launch Phoebus, an example is included with the below installation instructions to modify the behavior of the 'Home' button, used to bring up the custom 'PSC Launcher' panel using the Home button, instead of having to manually navigate to the file when needed.

4.2 Installing Phoebus

1. If not installed, install Java, e.g.:
`sudo apt update && sudo apt install openjdk-17-jdk`
2. Create the directory: `sudo mkdir /opt/phoebus`
`sudo chown $USER:epics /opt/phoebus`
`chmod 2775 /opt/phoebus`
3. Download Phoebus and extract to the `/opt/phoebus` directory
<https://github.com/ControlSystemStudio/phoebus/releases/download/v4.7.4/phoebus-4.7.4-linux.tar.gz>

```
cd /opt/phoebus
```

```
wget https://github.com/ControlSystemStudio/phoebus/releases/download/  
v4.7.4/phoebus-4.7.4-linux.tar.gz
```

```
tar -xvzf phoebus-4.7.4-linux.tar.gz -C . -strip-components=1
```

4. Make the shell script executable: `chmod +x phoebus.sh`
5. Alias Phoebus to run via command `run-phoebus`:

```
echo 'alias run-phoebus="/opt/phoebus/phoebus.sh"' » ~/.bashrc  
source ~/.bashrc
```
6. Now test! Try `run-phoebus` from a terminal.

Chapter 5

Prototyping and Development

5.1 Introduction

5.2 Testing the LTC2945

The LTC2945 testing was carried out using a Linear Tech eval board, DEMO CIRCUIT 1697A. Connections and modifications were made as follows:

- VIN: tied to Arduino +5V
- GND: tied to Arduino GND
- ADIN: Tied to GND
- SCL: Tied to Arduino Pin D8 I2C2 SCL
- SDAI/SDAO (JP2 set to TIE): Arduino Pin D9 I2C2 SDA
- VPU: Tied to Arduino +3v3
- VOUT: Float or tied to dummy load ($1\text{k}\Omega$ to $10\text{k}\Omega$)
- $20\text{m}\Omega$ shunt replaced with 17Ω shunt for testing with $\sim 5\text{k}\Omega$ load
- Installed $4.7\text{k}\Omega$ pull up on I2C lines via R4, R5 footprints on EVB

5.2.1 Calculating Shunt Value for Test

Using the 5V from the arduino as a source, and a load ranging 1k to 10k ohms, we calculate:

$$R_{L,min} = 1k\Omega$$

$$V_{shunt} = V_S \frac{R_S}{R_L + R_S}$$

$$R_S = \frac{V_{shunt} R_L}{V_s - V_{shunt}}$$

$$V_{shunt} = 0.1024V$$

$$R_{S,fullscale} = \frac{(0.1024V)(1k\Omega)}{5V - 0.1024V} = 20.91\Omega$$

Adding some headroom, say, 15%, and going to an E96 resistor value...

$$R_S = 0.85(20.91) = 17.8\Omega$$

Verifying for the high end of the range:

$$I_{min} = \frac{5V}{10k\Omega + 17.8\Omega} = 499.1\mu A$$

$$V_{shunt,min} = (499.1\mu A)(17.8\Omega) = 8.88mV$$

The shunt will range from 8.88mV to 87.44mV, within the ADC's usable range of 0V to 102.4mV.

5.2.2 Pictures of Evaluation Board, Display, and Phoebus Panels

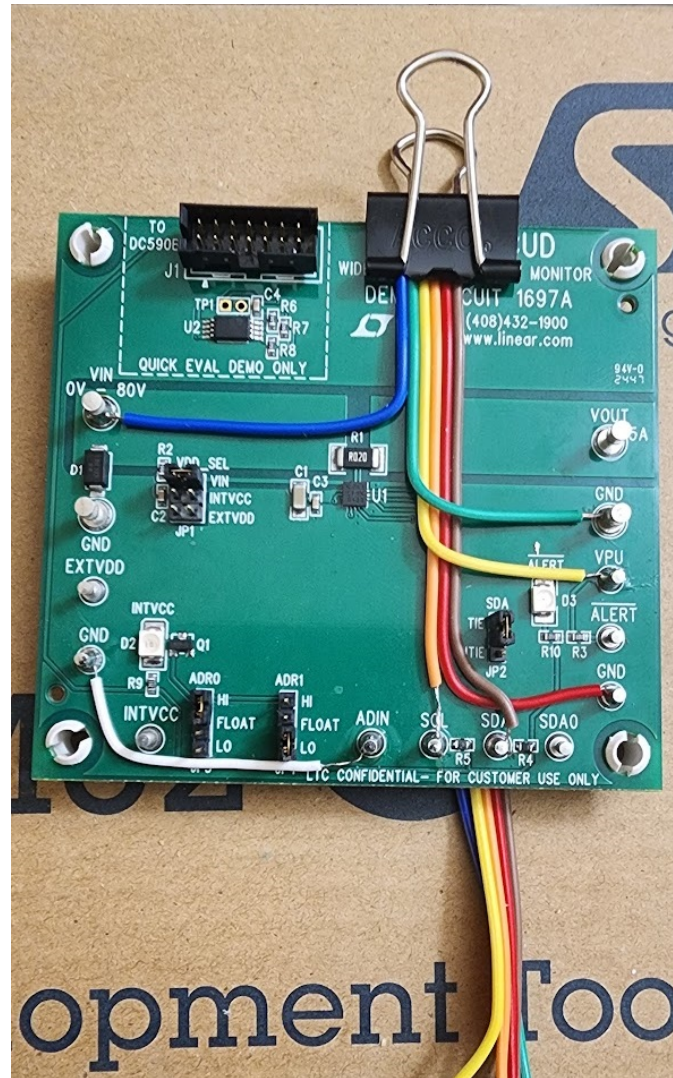


Figure 5.1: DEMO CIRCUIT 1697A Eval Board with Original Shunt, fly leads to connect to Arduino.



Figure 5.2: DEMO CIRCUIT 1697A Eval Board with 17Ω Shunt, fly leads to connect to Arduino, and $4.7k\Omega$ test load.

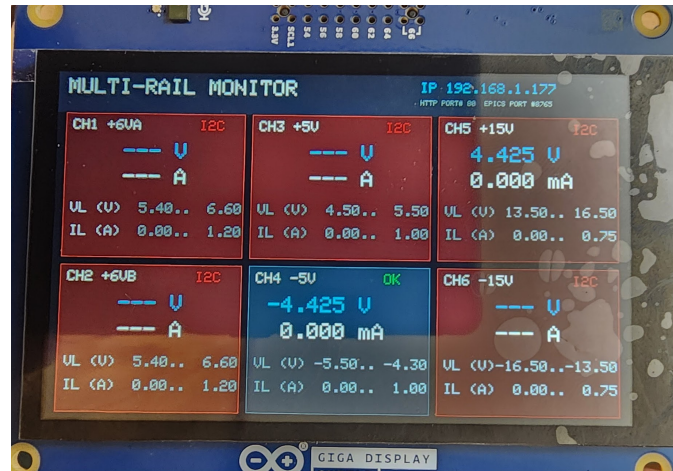


Figure 5.3: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm

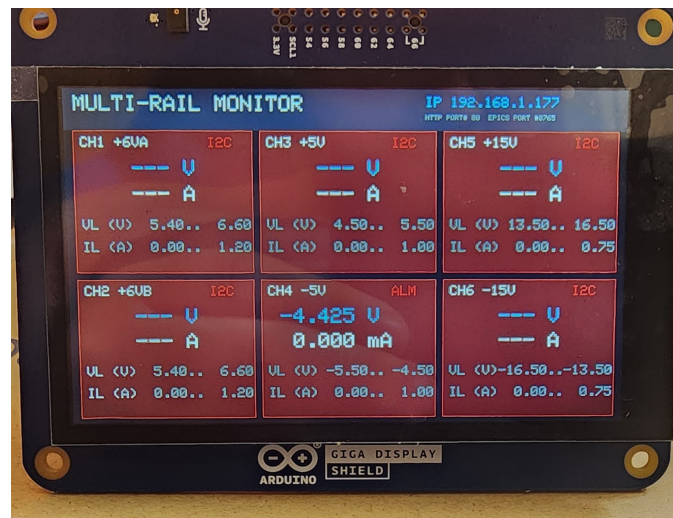


Figure 5.4: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, with UVL alarm, other channels I2C ALARM.

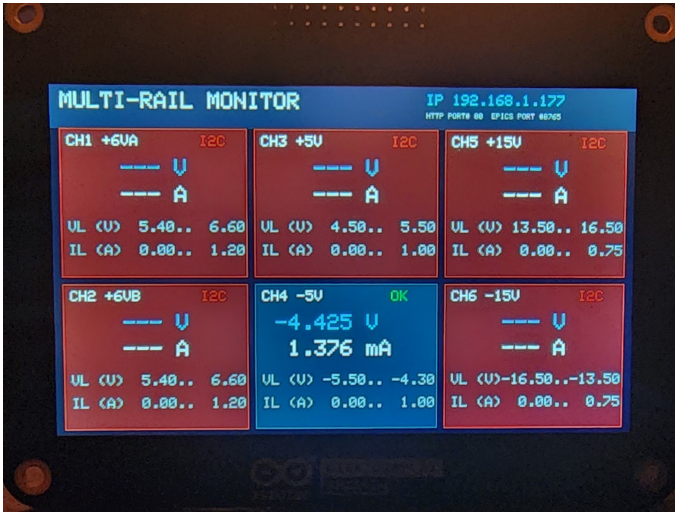


Figure 5.5: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm, 3.3kΩ Load



Figure 5.6: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm, 3.3kΩ Load



Figure 5.7: Arduino Giga with GIGA DISPLAY SHIELD connected to an LTC2945 eval board

